

Компьютерное зрение: от классических методов к глубокому обучению

Обработка изображений, Feature Extraction и Deep Learning

Лектор

Университет

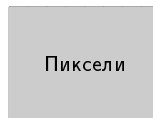
12 августа 2025 г.

1. Введение в Computer Vision
2. Основы обработки изображений
3. Фильтрация изображений
4. Классические методы Feature Extraction
5. Глубокое обучение для CV
6. Основные задачи CV
7. Обработка видео и анализ движения
8. Заключение

Что такое Computer Vision?

- **Computer Vision** — междисциплинарная область, которая изучает методы получения, обработки, анализа и понимания цифровых изображений
- Основные цели:
 - Автоматизация задач, выполняемых человеческой зрительной системой
 - Извлечение полезной информации из изображений
 - Интерпретация визуальных данных
- Применения: медицина, автономные транспортные средства, системы безопасности, робототехника

- Изображение = двумерная функция $f(x, y)$
- Дискретизация: $f(x, y) \rightarrow f[i, j]$
- Пиксель — минимальный элемент изображения
- Разрешение: количество пикселей (ширина $\times$ высота)
- Глубина цвета: количество бит на пиксель



- **RGB** (Red, Green, Blue):
 - Аддитивная модель
 - Каждый пиксель: (R, G, B) где $0 \leq R, G, B \leq 255$
- **HSV** (Hue, Saturation, Value):
 - Более интуитивное представление цвета
 - Hue: оттенок (0-360°), Saturation: насыщенность, Value: яркость
- **YUV/YCbCr**:
 - Y: яркость (luma), U,V: цветность (chroma)
 - Используется в видеокодеках
- **Lab**:
 - Перцептуально однородное пространство
 - L: светлота, a,b: цветовые компоненты

- **Аффинные преобразования:**

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (1)$$

- Виды преобразований:

- **Поворот:** $R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$
- **Масштабирование:** $S(s_x, s_y) = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$
- **Сдвиг:** $T(t_x, t_y) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$
- **Наклон (shear):** изменение углов

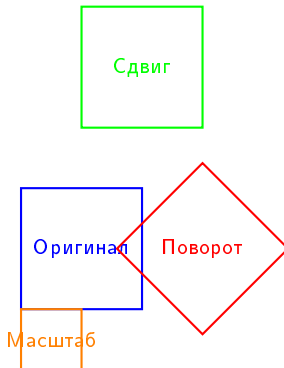
- **Проективные преобразования:** гомография (8 параметров)

Практические применения:

- Калибровка камеры: устранение дисторсий
- Панорамная съемка: сшивание изображений
- Дополненная реальность: наложение 3D объектов
- Медицинская визуализация: совмещение снимков

Пример поворота на 45°:

$$R(45) = \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} \quad (2)$$



- **Проблема:** при геометрических преобразованиях нужно определить значения пикселей в новых позициях
- **Методы интерполяции:**
 - **Nearest Neighbor:** $f(x, y) = f(\text{round}(x), \text{round}(y))$
 - **Билинейная:** взвешенное среднее 4 ближайших пикселей
 - **Бикубическая:** использует 16 ближайших пикселей
- **Билинейная интерполяция:**

$$f(x, y) = f(0, 0)(1 - x)(1 - y) + f(1, 0)x(1 - y) + f(0, 1)(1 - x)y + f(1, 1)xy \quad (3)$$

- **Trade-offs:**
 - Nearest Neighbor: быстро, но пикселизация
 - Билинейная: хороший баланс скорость/качество
 - Бикубическая: высокое качество, медленно

- **Свертка** (convolution):

$$g[i, j] = \sum_{m=-a}^a \sum_{n=-b}^b f[i - m, j - n] \cdot h[m, n] \quad (4)$$

где $h[m, n]$ — ядро фильтра

- **Сглаживающие фильтры:**

- Усредняющий: $h = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

- Гауссовский: $G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$

- **Выделение границ:**

- Собель: $S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$

- Лапласиан: $\nabla^2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$

- **Медианный фильтр:**

- Заменяет пиксель медианой в окрестности
- Эффективен против импульсного шума
- Сохраняет границы

- **Морфологические операции:**

- **Эрозия:** уменьшение объектов
- **Дилатация:** расширение объектов
- **Открытие:** эрозия + дилатация
- **Заккрытие:** дилатация + эрозия

- **Билатеральный фильтр:**

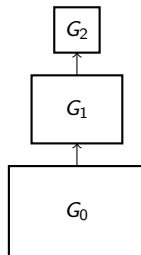
- Учитывает как пространственную, так и интенсивностную близость
- Сглаживает, сохраняя границы

- **Гауссова пирамида:**

- Последовательное сглаживание и субдискретизация
- Уровень $l + 1$: $G_{l+1} = \text{downsample}(\text{blur}(G_l))$
- Многомасштабное представление

- **Лапласова пирамида:**

- $L_l = G_l - \text{upsample}(G_{l+1})$
 - Разностное представление
 - Компактное кодирование
- Применения: сжатие, смешивание изображений, анализ текстур



Детекция границ - оператор Canny:

- 1 Гауссово сглаживание
- 2 Вычисление градиентов (Собель)
- 3 Подавление немаксимумов
- 4 Двойная пороговая фильтрация
- 5 Трассировка границ по связности

Параметры Canny:

- σ для Гаусса: 1.0-2.0
- Низкий порог: 50-100
- Высокий порог: 150-200

Пример кода (OpenCV):

```
Загрузка изображения img =  
cv2.imread('image.jpg', 0)  
Гауссово размытие blurred =  
cv2.GaussianBlur(img, (5,5), 1.4)  
Детекция границ Canny edges =  
cv2.Canny(blurred, 50, 150)  
Дилатация для утолщения линий kernel =  
np.ones((3,3))  
thick_edges = cv2.dilate(edges, kernel, iterations = 1)
```

- **Предобработка для детекции объектов:**

- Гауссово размытие для удаления шума
- Собель/Сэнны для выделения контуров
- Морфологические операции для очистки масок

- **Медицинская визуализация:**

- Усиление контраста с помощью unsharp masking
- Подавление шума в МРТ/КТ снимках
- Выделение структур органов

- **Обработка документов:**

- Бинаризация с адаптивным порогом
- Удаление шума сканирования
- Выпрямление перспективы

- **Фотография:**

- HDR тональное отображение
- Художественные эффекты
- Повышение резкости

- **Основная идея:** углы имеют большие градиенты в двух направлениях
- **Матрица структурного тензора:**

$$M = \sum_{(x,y) \in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \quad (5)$$

- **Harris response:**

$$R = \det(M) - k \cdot \text{trace}(M)^2 \quad (6)$$

где $k \approx 0.04 - 0.06$

- **Интерпретация:**
 - $R > 0$: угол
 - $R < 0$: граница
 - $|R|$ мало: однородная область

- **Основные свойства:**

- Инвариантность к масштабу и повороту
- Частичная инвариантность к освещению и аффинным преобразованиям

- **Алгоритм:**

- 1 Детекция экстремумов в DoG (Difference of Gaussians) пирамиде
- 2 Локализация ключевых точек с субпиксельной точностью
- 3 Определение ориентации на основе градиентов
- 4 Вычисление дескриптора (128-мерный вектор)

- **DoG:**

$$D(x, y, \sigma) = G(x, y, k\sigma) - G(x, y, \sigma) \quad (7)$$

SURF (Speeded-Up Robust Features):

- Быстрая аппроксимация SIFT
- Использует интегральные изображения
- Гессиан для детекции точек
- 64 или 128-мерный дескриптор

ORB (Oriented FAST and Rotated BRIEF):

- Быстрый детектор FAST
 - Бинарный дескриптор BRIEF
 - Инвариантность к повороту
 - Очень быстрый (real-time)
-
- **FAST**: Features from Accelerated Segment Test
 - **BRIEF**: Binary Robust Independent Elementary Features
 - Бинарные дескрипторы: быстрое сравнение через расстояние Хэмминга

- **Brute-Force Matching:**

- Сравнение каждого дескриптора с каждым
- Метрики: L2 (Euclidean), L1 (Manhattan), Hamming
- Сложность: $O(n \cdot m)$

- **FLANN (Fast Library for Approximate Nearest Neighbors):**

- KD-tree для низкоразмерных данных
- LSH (Locality Sensitive Hashing) для высокоразмерных
- Значительно быстрее при большом количестве features

- **Lowe's ratio test:**

- Отношение расстояний к первому и второму ближайшему соседу
- Порог обычно 0.7-0.8
- Уменьшает количество ложных совпадений

- **Проблема:** наличие выбросов (outliers) в совпадениях

- **Алгоритм RANSAC:**

- 1 Случайно выбрать минимальный набор точек
- 2 Построить модель (например, гомографию)
- 3 Подсчитать количество inliers
- 4 Повторить N раз, выбрать лучшую модель

- **Параметры:**

- Порог расстояния для inliers
- Количество итераций N
- Доля outliers ϵ

- **Количество итераций:**

$$N = \frac{\log(1 - p)}{\log(1 - (1 - \epsilon)^s)} \quad (8)$$

где p — вероятность успеха, s — размер выборки

Сшивание панорам:

- Детекция SIFT features
- Matching дескрипторов
- RANSAC для гомографии
- Наложение и смешивание

Калибровка камеры:

- Детекция углов шахматной доски
- RANSAC для отбора хороших изображений
- Вычисление внутренних параметров

Детекция плоскостей в 3D:

- RANSAC на облаках точек
- Модель плоскости: $ax + by + cz + d = 0$
- Применение в робототехнике

Пример: поиск линии:

```
for ;nrange(max_trials) : 2sample =
  random.sample(points, 2)
  Построить линию line = fit_line(sample)
  Подсчитать inliers inliers =
  count_nliers(points, line, threshold)
  if inliers > best_nliers : best_nliers =
  inliersbest_model = line
return best_model
```

Детектор	Скорость	Инвариантность	Размер дескр.	Точность	Примен
Harris	Быстрый	Поворот	-	Средняя	Tracki
SIFT	Медленный	Масштаб+поворот	128 float	Высокая	Panora
SURF	Средний	Масштаб+поворот	64 float	Высокая	Real-ti
ORB	Очень быстрый	Поворот	256 bit	Средняя	Mobi
FAST	Очень быстрый	Нет	-	Низкая	Vide

Рекомендации по выбору:

- **Высокая точность:** SIFT для сложных сцен
- **Real-time:** ORB для мобильных приложений
- **Видео:** FAST + оптический поток
- **Панорамы:** SIFT или SURF
- **Стерео:** SIFT или ORB в зависимости от требований

- **Ограничения классических методов:**

- Рукотворные features могут быть неоптимальными
- Сложность настройки параметров
- Ограниченная способность к обобщению

- **Преимущества Deep Learning:**

- Автоматическое изучение представлений
- End-to-end обучение
- Лучшая производительность на сложных задачах
- Масштабируемость с количеством данных

- **Ключевые факторы успеха:**

- Большие наборы данных (ImageNet, COCO)
- Вычислительная мощность (GPU)
- Эффективные архитектуры

- **Основные принципы:**

- **Локальная связность:** нейроны связаны только с локальной областью
- **Разделение весов:** одни и те же веса для всего изображения
- **Трансляционная инвариантность:** сдвиг входа = сдвиг выхода

- **Свертка 2D:**

$$Y[i, j] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} X[i + m, j + n] \cdot W[m, n] + b \quad (9)$$

- **Параметры свертки:**

- Размер ядра (kernel size): обычно 3×3 , 5×5
- Шаг (stride): как часто применяется ядро
- Отступ (padding): добавление нулей по краям

- **Сверточный слой (Conv2D):**

- Извлечение локальных features
- Множество фильтров = множество feature maps
- Размер выхода: $\frac{W-K+2P}{S} + 1$

- **Pooling слой:**

- Уменьшение пространственных размеров
- Max Pooling: max в окне
- Average Pooling: среднее в окне
- Инвариантность к небольшим сдвигам

- **Функции активации:**

- ReLU: $f(x) = \max(0, x)$ — наиболее популярная
- Leaky ReLU: $f(x) = \max(\alpha x, x)$
- Swish, GELU — современные альтернативы

- **VGG (2014):**
 - Простая архитектура: стековые 3×3 свертки
 - VGG-16, VGG-19
 - Много параметров (138M для VGG-16)
- **ResNet (2015):**
 - Остаточные связи: $y = F(x) + x$
 - Решает проблему затухающих градиентов
 - ResNet-50, ResNet-101, ResNet-152
- **MobileNet (2017):**
 - Depthwise separable convolutions
 - Оптимизация для мобильных устройств
 - Меньше параметров и вычислений

- **Основная идея:** применение Transformer архитектуры к изображениям
- **Алгоритм:**
 - 1 Разбиение изображения на патчи (например, 16×16)
 - 2 Линейная проекция патчей в векторы
 - 3 Добавление позиционных кодировок
 - 4 Поддача в стандартный Transformer encoder
- **Преимущества:**
 - Отсутствие индуктивных предпосылок CNN
 - Лучшая масштабируемость с размером данных
 - Глобальное внимание с первого слоя
- **Требования:** большие наборы данных для обучения

- **Проблемы классической детекции:**
 - Anchor boxes
 - Non-Maximum Suppression (NMS)
 - Ручная настройка гиперпараметров
- **DETR архитектура:**
 - CNN backbone (ResNet)
 - Transformer encoder-decoder
 - Множественные object queries
 - Bipartite matching loss
- **Преимущества:**
 - End-to-end обучение
 - Нет post-processing
 - Единая архитектура для детекции и сегментации

LeNet-5 (1998) - MNIST:

- 2 conv слоя + 3 FC
- 60K параметров
- Sigmoid активация
- Пионер CNN для распознавания цифр

AlexNet (2012) - ImageNet:

- 5 conv + 3 FC слоя
- 60M параметров
- ReLU + Dropout
- Прорыв в ImageNet

ResNet Residual Block:

$$y = F(x) + x \quad (10)$$

- Решает проблему vanishing gradients
- Позволяет обучать сети 100+ слоев

Сравнение производительности:

Модель	Топ-1 (%)	Параметры
AlexNet	42.9	60M
VGG-16	28.1	138M
ResNet-50	23.9	25M
ResNet-152	21.3	60M
DenseNet-201	22.0	20M
EfficientNet-B7	15.7	66M

Ключевые инновации:

- **Batch Normalization**: стабилизация обучения
- **Skip connections**: глубокие сети
- **Depthwise convolutions**: эффективность
- **Attention**: фокус на важном
- **Neural Architecture Search**: автоматизация

Стратегии Transfer Learning:

1 Feature extraction:

- Заморозить CNN backbone
- Обучить только классификатор
- Мало данных, похожая задача

2 Fine-tuning:

- Разморозить последние слои
- Низкий learning rate
- Больше данных

3 Full training:

- Обучить всю сеть
- Много данных, новая задача

Пример кода (PyTorch):

```
Загрузить предобученную модель model =  
models.resnet50(pretrained=True)  
Заморозить параметры for param in  
model.parameters(): param.requires_grad = False  
Заменить последний слой  
num_classes = 10  
model.fc = nn.Linear(model.fc.in_features, num_classes)  
Для fine-tuning разморозить последние слои  
for param in model.layer4.parameters():  
param.requires_grad = True
```

Советы:

- Начинать с малого LR (1e-4)
- Использовать разные LR для слоев
- Аугментация данных важна

- **Задача:** определить, к какому классу принадлежит изображение
- **Архитектура:**
 - CNN backbone
 - Global Average Pooling или Flatten
 - Fully Connected слой
 - Softmax для многоклассовой классификации
- **Loss функции:**
 - Cross-entropy: $L = - \sum_i y_i \log(\hat{y}_i)$
 - Focal loss: решает проблему дисбаланса классов
- **Метрики:** Accuracy, Top-k accuracy, F1-score, AUC-ROC
- **Техники улучшения:** Data augmentation, Transfer learning, Ensemble

- **Задача:** найти и классифицировать все объекты на изображении
- **Выход:** список bounding boxes с классами и confidence scores
- **Типы архитектур:**
 - **Двухстадийные:** Region proposals → классификация
 - **Одностадийные:** прямое предсказание boxes и классов
 - **Transformer-based:** использование механизма внимания
- **Основные вызовы:**
 - Различные размеры объектов
 - Перекрывающиеся объекты
 - Дисбаланс классов
 - Real-time inference

- **Архитектура:**

- ① **CNN backbone:** извлечение feature maps
- ② **RPN (Region Proposal Network):** генерация candidate boxes
- ③ **ROI Pooling:** извлечение features для каждого proposal
- ④ **Классификация и регрессия:** финальные предсказания

- **RPN:**

- Anchor boxes разных размеров и соотношений сторон
- Бинарная классификация (объект/фон)
- Регрессия координат bounding box

- **Loss функция:**

$$L = L_{cls} + \lambda L_{reg} \quad (11)$$

- **Преимущества:** высокая точность

- **Недостатки:** медленная inference

- **YOLO (You Only Look Once):**
 - Разделение изображения на сетку $S \times S$
 - Каждая ячейка предсказывает B bounding boxes
 - Прямое предсказание координат и классов
- **Выходной тензор:** $S \times S \times (B \times 5 + C)$
 - $5 = (x, y, w, h, confidence)$
 - C = количество классов
- **SSD (Single Shot MultiBox Detector):**
 - Множественные feature maps разных размеров
 - Default boxes на каждой позиции
 - Лучше для объектов разных размеров
- **Преимущества:** высокая скорость
- **Недостатки:** сложности с маленькими объектами

- **Задача:** классификация каждого пикселя изображения
- **U-Net:**
 - Encoder-decoder архитектура
 - Skip connections между encoder и decoder
 - Популярна в медицинской сегментации
- **DeepLab:**
 - Dilated/Atrous convolutions
 - Atrous Spatial Pyramid Pooling (ASPP)
 - Захват контекста разных масштабов
- **Loss функции:**
 - Cross-entropy loss
 - Dice loss: $1 - \frac{2|A \cap B|}{|A| + |B|}$
 - Focal loss для дисбаланса классов
- **Метрики:** IoU, Dice coefficient, pixel accuracy

- **Задача:** детекция + сегментация каждого экземпляра объекта

- **Архитектура Mask R-CNN:**

- Основа: Faster R-CNN
- Дополнительная ветвь для предсказания масок
- ROI Align вместо ROI Pooling

- **ROI Align:**

- Устраняет квантование в ROI Pooling
- Билинейная интерполяция
- Сохраняет точность локализации для масок

- **Multi-task loss:**

$$L = L_{cls} + L_{box} + L_{mask} \quad (12)$$

- **Применения:** медицина, автономные машины, робототехника

Модель	mAP (%)	FPS	Стадии	Anchors	NMS
R-CNN	58.5	0.05	2	Да	Да
Fast R-CNN	70.0	0.5	2	Да	Да
Faster R-CNN	73.2	7	2	Да	Да
YOLO v1	63.4	45	1	Нет	Да
SSD	74.3	59	1	Да	Да
YOLO v3	57.9	20	1	Да	Да
YOLO v5	64.1	140	1	Да	Да
DETR	42.0	28	1	Нет	Нет

Тренды развития:

- **Скорость vs Точность:** YOLO для real-time, R-CNN для точности
- **Anchor-free:** FCOS, CenterNet устраняют anchor boxes
- **Transformer-based:** DETR, Deformable DETR
- **Efficiency:** EfficientDet оптимизирует accuracy/speed trade-off
- **3D Detection:** для автономных машин и робототехники

Медицинская диагностика:

- **Рентген**: детекция пневмонии
- **MPT**: сегментация опухолей мозга
- **Дерматология**: классификация меланомы
- **Офтальмология**: детекция диабетической ретинопатии

Автономные транспортные средства:

- **Детекция объектов**: машины, пешеходы, знаки
- **Сегментация**: дорога, тротуар, разметка
- **Depth estimation**: LiDAR + камера
- **Tracking**: траектории других участников

Производство и качество:

- **Дефектоскопия**: поиск трещин, царапин
- **Сортировка**: классификация продукции
- **Измерения**: контроль размеров деталей
- **Сборка**: проверка правильности установки

Розничная торговля:

- **Касса самообслуживания**: распознавание товаров
- **Инвентаризация**: подсчет товаров на полках
- **Поведение покупателей**: анализ маршрутов
- **Рекомендации**: на основе визуального поиска

Геометрические преобразования:

- Поворот (-15° до $+15^\circ$)
- Масштабирование (0.8-1.2)
- Сдвиг (до 10% от размера)
- Отражение (horizontal flip)
- Искажение перспективы

Цветовые преобразования:

- Изменение яркости ($\pm 20\%$)
- Изменение контраста (0.8-1.2)
- Изменение насыщенности
- Изменение оттенка ($\pm 10^\circ$)
- Гамма-коррекция

Современные методы:

- **Mixup**: $\lambda x_1 + (1 - \lambda)x_2$
- **CutMix**: замена регионов
- **AutoAugment**: автоматический поиск политик
- **RandAugment**: случайная магнитуда
- **AugMix**: смешивание аугментаций

Пример кода:

```
transform = A.Compose([ A.RandomRotate90(),  
                        A.Flip(), A.Transpose(), A.OneOf([  
                        A.GaussNoise(), A.GaussianBlur(),  
                        A.MotionBlur(), ], p=0.2), A.Normalize() ])
```

- **Определение:** паттерн видимого движения объектов между кадрами
- **Основное уравнение:**

$$I(x, y, t) = I(x + dx, y + dy, t + dt) \quad (13)$$

- **Linearization:**

$$I_x u + I_y v + I_t = 0 \quad (14)$$

где (u, v) — optical flow, I_x, I_y, I_t — градиенты

- **Проблема апертуры:** одно уравнение, две неизвестные
- **Предположения:**
 - Яркостное постоянство
 - Пространственная когерентность
 - Временная согласованность

- **Lucas-Kanade:**

- Локальный метод
- Предположение о постоянстве flow в окрестности
- Решение системы уравнений методом наименьших квадратов
- Хорош для отслеживания sparse features

- **Farneback:**

- Dense optical flow
- Аппроксимация окрестности квадратичными полиномами
- Вычисляет flow для каждого пикселя

- **FlowNet (глубокое обучение):**

- End-to-end CNN для optical flow
- FlowNetS: простая архитектура
- FlowNetC: correlation layer для сопоставления features

- **Задача:** поддержание траектории объекта в видеопоследовательности
- **Kalman Filter:**
 - Байесовский фильтр для линейных систем
 - Prediction: $\mathbf{x}_{k|k-1} = \mathbf{F}_k \mathbf{x}_{k-1|k-1}$
 - Update: $\mathbf{x}_{k|k} = \mathbf{x}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H}_k \mathbf{x}_{k|k-1})$
 - Подходит для линейного движения
- **SORT (Simple Online and Realtime Tracking):**
 - Kalman filter + Hungarian algorithm
 - Использует только bounding box информацию
 - Быстрый, но простой
- **DeepSORT:**
 - SORT + appearance features от CNN
 - Cosine distance для association
 - Более робастный к окклюзиям

- **Задача:** классификация действий в видео
- **Подходы:**
 - **3D CNN:** прямое расширение 2D CNN на временное измерение
 - **Two-stream networks:** RGB + optical flow streams
 - **RNN-based:** LSTM поверх CNN features
 - **3D ResNet:** остаточные связи в 3D
- **Temporal modeling:**
 - Фиксированное количество кадров
 - Temporal pooling
 - Attention mechanisms
- **Датасеты:** UCF-101, HMDB-51, Kinetics, ActivityNet
- **Применения:** видеонаблюдение, спортивная аналитика, HCI

Система видеонаблюдения:

- 1 Детекция движения: разность кадров
- 2 Детекция объектов: YOLO на каждом кадре
- 3 Tracking: DeepSORT для траекторий
- 4 Распознавание действий: 3D CNN
- 5 Анализ аномалий: автоэнкодеры

Спортивная аналитика:

- Трекинг игроков и мяча
- Распознавание тактических паттернов
- Автоматическое создание хайлайтов
- Анализ биомеханики движений

Пример pipeline:

```
Инициализация detector = YOLO('yolov5s.pt')
tracker = DeepSORT() action_model = load_action_model()
cap = cv2.VideoCapture('video.mp4')
while True: ret, frame = cap.read() if not ret: break
Детекция detections = detector(frame)
Трекинг tracks = tracker.update(detections)
Накопление кадров для
действий for track in tracks:
track.add_frame(frame) if len(track.frames) == 16:
: action = action_model.predict(track.frames) track.action = action
Визуализация draw_tracks_and_actions(frame, tracks)
```

- **Технические вызовы:**

- **Реального времени обработка:** оптимизация для edge устройств
- **Малое количество данных:** few-shot и zero-shot learning
- **Робастность:** adversarial examples и domain adaptation
- **Объяснимость:** интерпретируемые модели и GradCAM
- **Privacy:** федеративное обучение и дифференциальная приватность

- **Новые направления:**

- **Multimodal AI:** интеграция текста, аудио, видео
- **Neural Radiance Fields (NeRF):** 3D реконструкция
- **Diffusion Models:** генерация изображений
- **Foundation Models:** большие предобученные модели
- **Embodied AI:** робототехника с компьютерным зрением

- **Этические аспекты:**

- Bias в данных и алгоритмах
- Surveillance и приватность
- Deepfakes и дезинформация
- Прозрачность принятия решений

- **Vision Transformers:** постепенное вытеснение CNN
- **Self-supervised learning:**
 - Обучение без размеченных данных
 - Contrastive learning (SimCLR, MoCo)
 - Masked image modeling (MAE)
- **Multimodal learning:**
 - CLIP: joint vision-language representation
 - DALL-E: text-to-image generation
 - GPT-4V: multimodal understanding
- **Neural Architecture Search (NAS):** автоматический поиск архитектур
- **Edge deployment:** оптимизация для мобильных устройств

- **Выбор архитектуры:**
 - Accuracy vs Speed trade-off
 - Размер модели и память
 - Доступные данные для обучения
- **Data augmentation:**
 - Геометрические преобразования
 - Цветовые трансформации
 - Mixup, CutMix
- **Transfer learning:**
 - Pre-trained модели на ImageNet
 - Fine-tuning vs Feature extraction
 - Domain adaptation
- **Evaluation:** правильная оценка на тестовых данных

- Computer Vision прошло путь от простых фильтров до сложных нейронных сетей
- **Ключевые компоненты:**
 - Обработка изображений и feature extraction
 - Глубокое обучение и CNN
 - Специализированные архитектуры для разных задач
 - Анализ видео и temporal modeling
- **Будущие направления:**
 - Более эффективные архитектуры
 - Лучшее понимание 3D и temporal структуры
 - Интеграция с другими модальностями
 - Объяснимость и робастность
- CV продолжает быстро развиваться и находить новые применения

Книги и курсы:

- [Computer Vision: Algorithms and Applications](#) - Szeliski
- [Deep Learning](#) - Goodfellow, Bengio, Courville
- [CS231n Stanford](#) - CNN для визуального распознавания
- [CS231A Stanford](#) - Computer Vision: From 3D Reconstruction to Recognition
- [Fast.ai](#) - практический deep learning

Библиотеки и фреймворки:

- [OpenCV](#): классическое CV
- [PyTorch/TensorFlow](#): deep learning
- [torchvision](#): предобученные модели
- [Detectron2](#): Facebook AI Research
- [MMDetection](#): OpenMMLab toolkit

Датасеты для практики:

- [MNIST](#): классификация цифр
- [CIFAR-10/100](#): классификация объектов
- [ImageNet](#): крупномасштабная классификация
- [COCO](#): детекция и сегментация
- [Open Images](#): Google dataset
- [Cityscapes](#): автономные машины

Соревнования и сообщества:

- [Kaggle](#): соревнования по CV
- [Papers with Code](#): актуальные исследования
- [arXiv](#): препринты статей
- [GitHub](#): открытый код
- [Reddit r/MachineLearning](#): обсуждения

Спасибо за внимание!

Вопросы?

Computer Vision - это увлекательная область, которая продолжает стремительно развиваться.

Практикуйтесь, экспериментируйте и следите за новыми достижениями!